



Don't share databases unless you really need to. And even then do everything you can to avoid it. In my opinion, sharing databases is one of the worst things you can do if you're trying to achieve independent deployability.

As discussed in the previous section, we want to think of our services as end-to-end slices of business functionality that, where appropriate, encapsulate user interface (UI), business logic, and data. This is because we want to reduce the effort needed to change business-related functionality. The encapsulation of data and behavior in this way gives us high cohesion of business functionality. By hiding the database that backs our service, we also ensure that we reduce coupling. We come back to coupling and cohesion in [Chapter 2](#).

Size

“How big should a microservice be?” is one of the most common questions I hear. Considering the word “micro” is right there in the name, this comes as no surprise. However, when you get into what makes microservices work as a type of architecture, the concept of size is actually one of the least interesting aspects.

How do you measure size? By counting lines of code? That doesn't make much sense to me. Something that might require 25 lines of code in Java could be written in 10 lines of Clojure. That's not to say Clojure is better or worse than Java; some languages are simply more expressive than others.

James Lewis, technical director at Thoughtworks, has been known to say that “a microservice should be as big as my head.” At first glance, this doesn't seem terribly helpful. After all, how big is James's head exactly? The rationale behind this statement is that a microservice should be kept to the size at which it can be easily understood. The challenge, of course, is that different people's ability to understand something isn't always the same, and as such you'll need to make your own judgment regarding what size works for you. An experienced team may be able to better manage a larger codebase than another team could. So perhaps it would be better to read James's quote here as “a microservice should be as big as *your* head.”

I think the closest I get to “size” having any meaning in terms of microservices is something that Chris Richardson, the author of *Microservice Patterns* (Manning Publications), once said—the goal of microservices is to have “as small an interface as possible.” That aligns with the concept of information hiding again, but it does represent an attempt to find meaning in the term “microservices” that wasn't there initially. When the term was first used to define these architectures, the focus, at least initially, was not specifically on the size of the interfaces.

Ultimately, the concept of size is highly contextual. Speak to a person who has worked on a system for 15 years, and they'll feel that their system with 100,000 lines

of code is really easy to understand. Ask the opinion of someone brand-new to the project, and they'll feel it's much too big. Likewise, ask a company that has just embarked on its microservice transition and has perhaps 10 or fewer microservices, and you'll get a different answer than you would from a similar-sized company for which microservices have been the norm for many years and that now has hundreds.

I urge people not to worry about size. When you are first starting out, it's much more important that you focus on two key things. First, how many microservices can you handle? As you have more services, the complexity of your system will increase, and you'll need to learn new skills (and perhaps adopt new technology) to cope with this. A move to microservices will introduce new sources of complexity, with all the challenges this can bring. It's for this reason that I am a strong advocate for incremental migration to a microservice architecture. Second, how do you define microservice boundaries to get the most out of them, without everything becoming a horribly coupled mess? These topics are much more important to focus on when you start your journey.

Flexibility

Another quote from James Lewis is that “microservices buy you options.” Lewis was being deliberate with his words—they *buy* you *options*. They have a cost, and you must decide whether the cost is worth the options you want to take up. The resulting flexibility on a number of axes—organizational, technical, scale, robustness—can be incredibly appealing.

We don't know what the future holds, so we'd like an architecture that can theoretically help us solve whatever problems we might face down the road. Finding a balance between keeping your options open and bearing the cost of architectures like this can be a real art.

Think of adopting microservices as less like flipping a switch, and more like turning a dial. As you turn up the dial, and you have more microservices, you have increased flexibility. But you likely ramp up the pain points too. This is yet another reason I strongly advocate incremental adoption of microservices. By turning up the dial gradually, you are better able to assess the impact as you go, and to stop if required.

Alignment of Architecture and Organization

MusicCorp, an ecommerce company that sells CDs online, uses the simple three-tiered architecture shown earlier in [Figure 1-2](#). We've decided to move MusicCorp kicking and screaming into the 21st century, and as part of that, we're assessing the existing system architecture. We have a web-based UI, a business logic layer in the form of a monolithic backend, and data storage in a traditional database. These layers, as is common, are owned by different teams. We'll be coming back to the trials and tribulations of MusicCorp throughout the book.

We want to make a simple update to our functionality: we want to allow our customers to specify their favorite genre of music. This update requires us to change the UI to show the genre choice UI, the backend service to allow for the genre to be surfaced to the UI and for the value to be changed, and the database to accept this change. These changes will need to be managed by each team and deployed in the correct order, as outlined in [Figure 1-3](#).

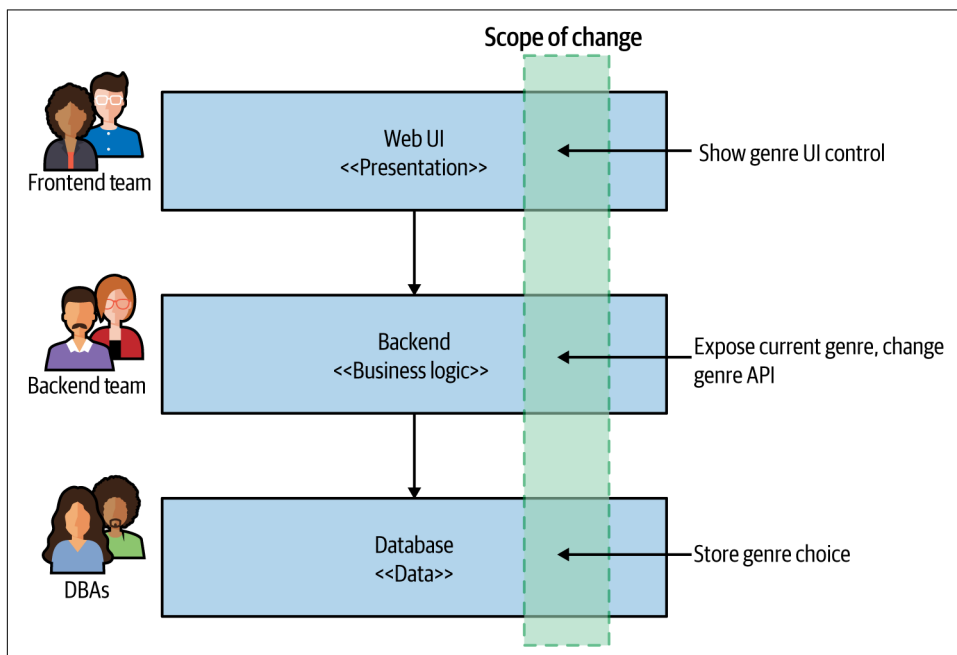


Figure 1-3. Making a change across all three tiers is more involved

Now this architecture isn't bad. All architecture ends up getting optimized around a set of goals. Three-tiered architecture is so common partly because it is universal—everyone has heard about it. So the tendency to pick a common architecture that you might have seen elsewhere is often one reason we keep seeing this pattern. But I think the biggest reason we see this architecture again and again is because it is based on how we organize our teams.

The now famous Conway's law states the following:

Organizations which design systems...are constrained to produce designs which are copies of the communication structures of these organizations.

—Melvin Conway, "[How Do Committees Invent?](#)"

The three-tiered architecture is a good example of this law in action. In the past, the primary way IT organizations grouped people was in terms of their core competency:

database admins were in a team with other database admins; Java developers were in a team with other Java developers; and frontend developers (who nowadays know exotic things like JavaScript and native mobile application development) were in yet another team. We group people based on their core competency, so we create IT assets that can be aligned to those teams.

That explains why this architecture is so common. It's not bad; it's just optimized around one set of forces—how we traditionally grouped people, around familiarity. But the forces have changed. Our aspirations around our software have changed. We now group people in poly-skilled teams to reduce handoffs and silos. We want to ship software much more quickly than ever before. That is driving us to make different choices about the way we organize our teams, so that we organize them in terms of the way we break our systems apart.

Most changes that we are asked to make to our system relate to changes in business functionality. But in [Figure 1-3](#), our business functionality is in effect spread across all three tiers, increasing the chance that a change in functionality will cross layers. This is an architecture that has high cohesion of related technology but low cohesion of business functionality. If we want to make it easier to make changes, instead we need to change how we group code, choosing cohesion of business functionality rather than technology. Each service may or may not end up containing a mix of these three layers, but that is a local service implementation concern.

Let's compare this with a potential alternative architecture, illustrated in [Figure 1-4](#). Rather than a horizontally layered architecture and organization, we instead break down our organization and architecture along vertical business lines. Here we see a dedicated team that has full-end-to-end responsibility for making changes to aspects of the customer profile, which ensures that the scope of change in this example is limited to one team.

As an implementation, this could be achieved through a single microservice owned by the profile team that exposes a UI to allow customers to update their information, with state of the customer also stored within this microservice. The choice of a favorite genre is associated with a given customer, so this change is much more localized. In [Figure 1-5](#), we also show the list of available genres being fetched from a `Catalog` microservice, something that would likely already be in place. We also see a new `Recommendation` microservice accessing our favorite genre information, something that could easily follow in a subsequent release.